

Laboratório 2 - Sistemas Operacionais

Aluna: Maria Eduarda da Nóbrega

Disciplina: Sistemas Operacionais — UEPB

Data: 01/10/2026

Os PIDs que aparecem nas saídas e nos diagramas vêm de execuções reais (WSL/Ubuntu) e mudam a cada execução. Os códigos comentados completos estão nos arquivos `fork.c`, `fork-execve.c` e `fork-print.c`, entregues junto com este documento.

Exercício 1 — `fork.c`

1. Análise e comentários do código

O programa demonstra a criação de um processo com a chamada de sistema `fork()`. O `fork()` duplica o processo que o chamou: o novo processo (filho) recebe uma cópia do código, dos dados, da pilha e dos registradores, e continua executando a partir da instrução seguinte ao `fork()`. O valor retornado permite distinguir quem é quem: negativo (erro), zero (filho) ou o PID do filho (pai).

```
int main ()
{
    int retval ; // retorno do fork(): <0 erro, 0 filho, >0 pai (PID do filho)

    // executado só pelo processo original
    printf ("Ola, sou o processo %5d\n", getpid()) ;

    // cria o filho: a partir daqui existem DOIS processos
    retval = fork () ;

    // executado pelos dois: cada um imprime seu retval, seu PID e o PID do pai
    printf ("[retval: %5d] sou %5d, filho de %5d\n", retval, getpid(), getppid()) ;

    if ( retval < 0 ) // erro no fork()
    {
        perror ("Erro") ; // mostra o motivo do erro
        exit (1) ; // termina com código de falha
    }
    else
    {
        if ( retval > 0 ) // PAI
            wait (0) ; // bloqueia até o filho terminar (espera)
        else // FILHO
            sleep (5) ; // dorme 5 s (espera)

        // executado pelos dois: o filho após o sleep; o pai após o término do filho
        printf ("Tchau de %5d!\n", getpid()) ;
        exit (0) ;
    }
}
```

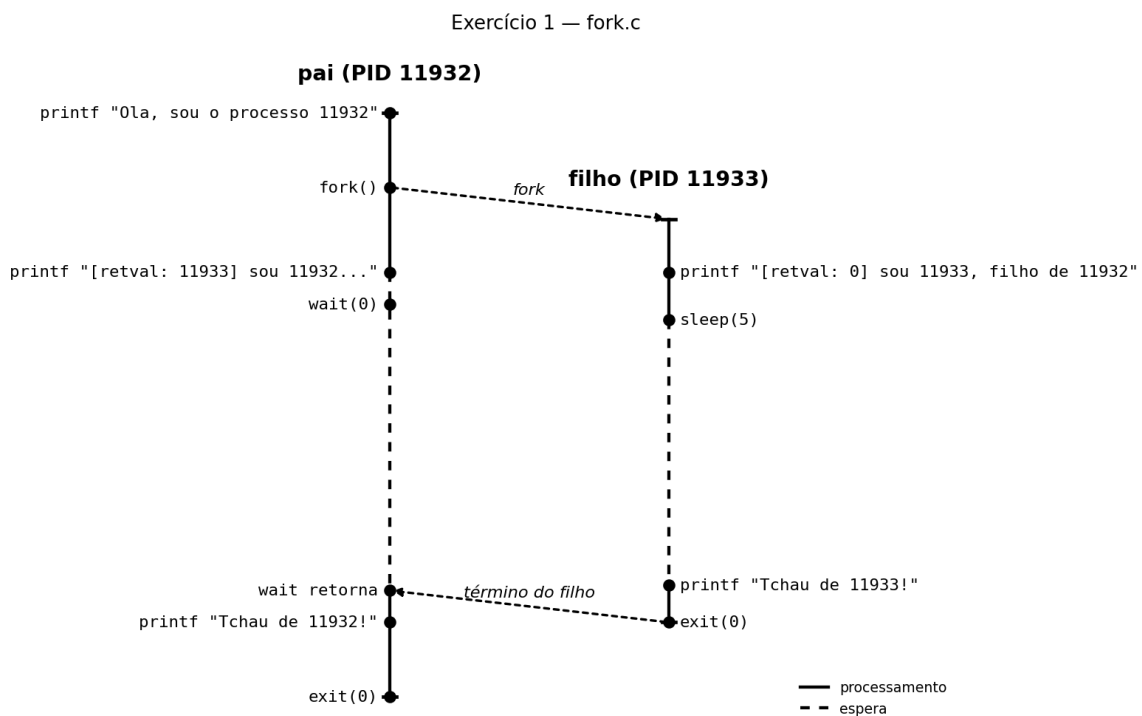
- A primeira mensagem é impressa uma única vez, pois o `fork` ainda não aconteceu.
- A segunda mensagem é impressa duas vezes. Para o pai, `retval` é o PID do filho e `getppid()` é o PID do shell; para o filho, `retval` é 0 e `getppid()` é o PID do pai.

- A ordem entre as duas mensagens após o fork depende do escalonador (não é determinística).
- O pai chama wait(0) e fica bloqueado até o filho terminar; o filho dorme 5 segundos. Assim, o "Tchau" do filho sempre sai antes do "Tchau" do pai.

Saída obtida no terminal:

```
Ola, sou o processo 11932
[retval: 11933] sou 11932, filho de 8757
[retval: 0] sou 11933, filho de 11932
Tchau de 11933!
Tchau de 11932!
```

2. Diagrama de tempo



Exercício 2 — fork-execve.c

1. Análise e comentários do código

Este programa é parecido com o anterior, mas o filho, em vez de dormir, substitui seu programa por outro usando execve(). O execve(caminho, argv, envp) carrega um novo executável no mesmo processo (o PID não muda): código, dados e pilha são substituídos pelos do novo programa. Por isso, quando tem sucesso, execve nunca retorna.

```
int main (int argc, char *argv[], char *envp[]) // args e ambiente, repassados ao execve
{
    int retval ;                               // retorno do fork()

    printf ("Ola, sou o processo %5d\n", getpid()) ; // só o processo original
    retval = fork () ;                          // cria o filho
```

```

printf ("[retval: %5d] sou %5d, filho de %5d\n",
        retval, getpid(), getppid()) ;           // pai e filho

if ( retval < 0 )           // erro no fork
{
    perror ("Erro: ") ;
    exit (1) ;
}
else
    if ( retval > 0 )       // PAI
        wait (0) ;         // espera o filho terminar
    else                   // FILHO
    {
        execve ("/bin/date", argv, envp) ; // troca a imagem do filho pelo /bin/date
        perror ("Erro") ;                 // só executa se o execve FALHAR
    }

printf ("Tchau de %5d!\n", getpid()) ;         // execve ok: só o pai chega aqui
exit (0) ;
}

```

- Com execve bem-sucedido, o filho vira o programa date: imprime a data e a hora e termina. As linhas após o execve (perror, printf "Tchau", exit) nunca são executadas no filho.
- O pai, bloqueado no wait(0), acorda quando o date termina, imprime "Tchau" e termina. Logo, só há UM "Tchau" na saída.

Saída obtida no terminal:

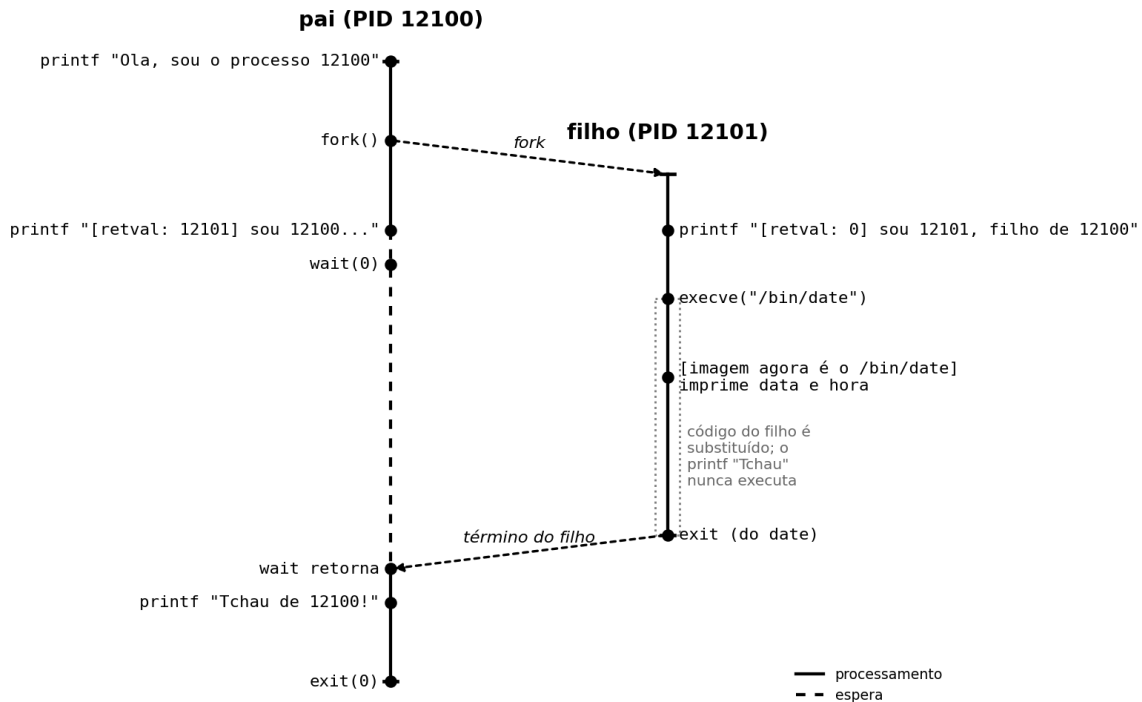
```

Ola, sou o processo 12100
[retval: 12101] sou 12100, filho de 8757
[retval:    0] sou 12101, filho de 12100
Thu Oct 1 16:47:47 -03 2026
Tchau de 12100!

```

2. Diagrama de tempo

Exercício 2 (item 2) — fork-execve.c com /bin/date existente

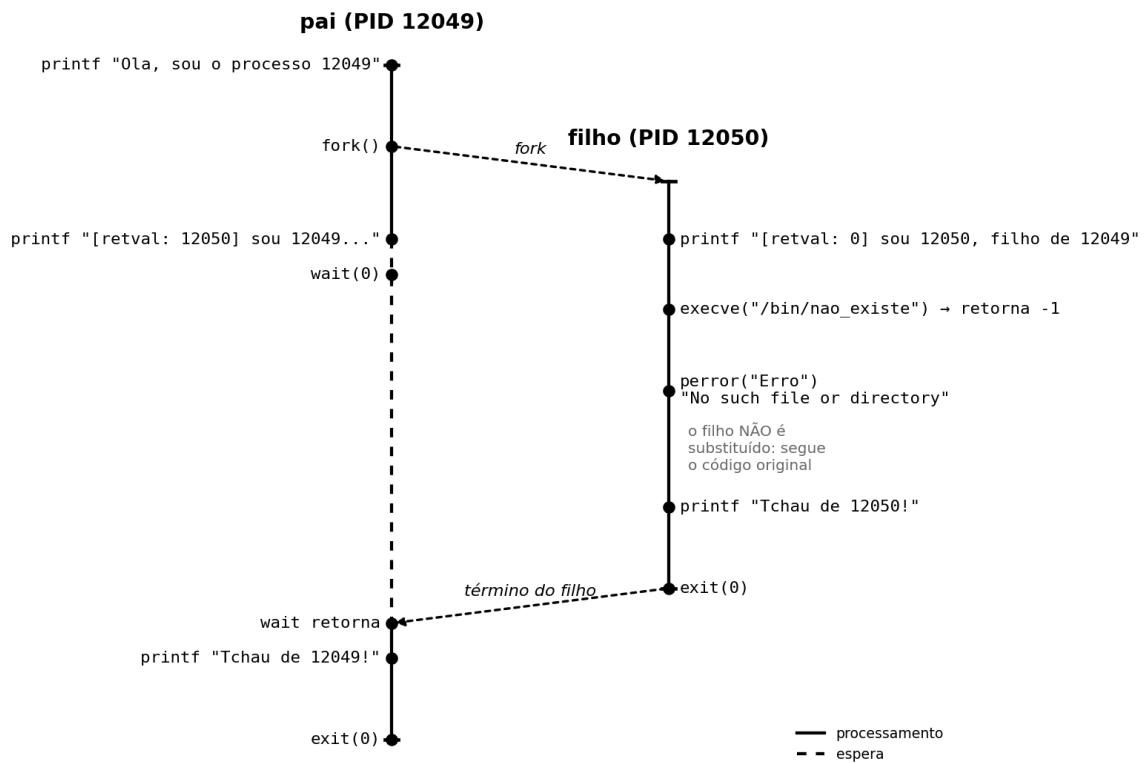


3. O que ocorre se o programa executado pelo execve não existir?

Se o arquivo não existir (ou não puder ser executado), o `execve` falha: retorna -1, define `errno` (`ENOENT`, "No such file or directory") e NÃO substitui a imagem do processo. O filho então continua executando o código original: chama `perror("Erro")`, que imprime a mensagem de erro, sai do bloco `if/else`, imprime "Tchau de <PID do filho>!" e termina com `exit(0)`. O pai, que aguardava no `wait(0)`, acorda e imprime seu próprio "Tchau". Portanto, nesse caso aparecem DOIS "Tchau" (um de cada processo) e a mensagem de erro. Foi testado trocando `/bin/date` por `/bin/nao_existe` e executando novamente.

```
Ola, sou o processo 12049
[retval: 12050] sou 12049, filho de 8757
[retval: 0] sou 12050, filho de 12049
Erro: No such file or directory
Tchau de 12050!
Tchau de 12049!
```

Exercício 2 (item 3) — execve falha (programa inexistente)



Exercício 3 — fork-print.c

1. Análise e comentários do código

O programa mostra que pai e filho possuem espaços de endereçamento independentes: após o fork, a variável x existe nos dois processos, mas são cópias distintas, e alterar uma não afeta a outra.

```
int main ()
{
    int retval, x ;
    x = 0 ;                // x vale 0 antes do fork
    retval = fork () ;     // o filho recebe uma CÓPIA da memória (x = 0 também)

    printf ("No processo %5d x vale %d\n", getpid(), x) ;    // pai e filho: x = 0

    if ( retval < 0 )      // erro
    {
        perror ("Erro") ;
        exit (1) ;
    }
    else
    {
        if ( retval > 0 )  // PAI
        {
            x = 0 ;       // atribui 0 ao x do pai (continua 0)
            wait (0) ;     // espera o filho terminar
        }
        else              // FILHO
        {
            x++ ;         // incrementa só o x do filho: passa a 1
        }
    }
}
```

```

    sleep (5) ;           // dorme 5 s
}

printf ("No processo %5d x vale %d\n", getpid(), x) ;    // filho: 1; pai: 0
exit (0) ;
}

```

Saída obtida no terminal (o filho imprime a mensagem final primeiro, porque o pai está no wait):

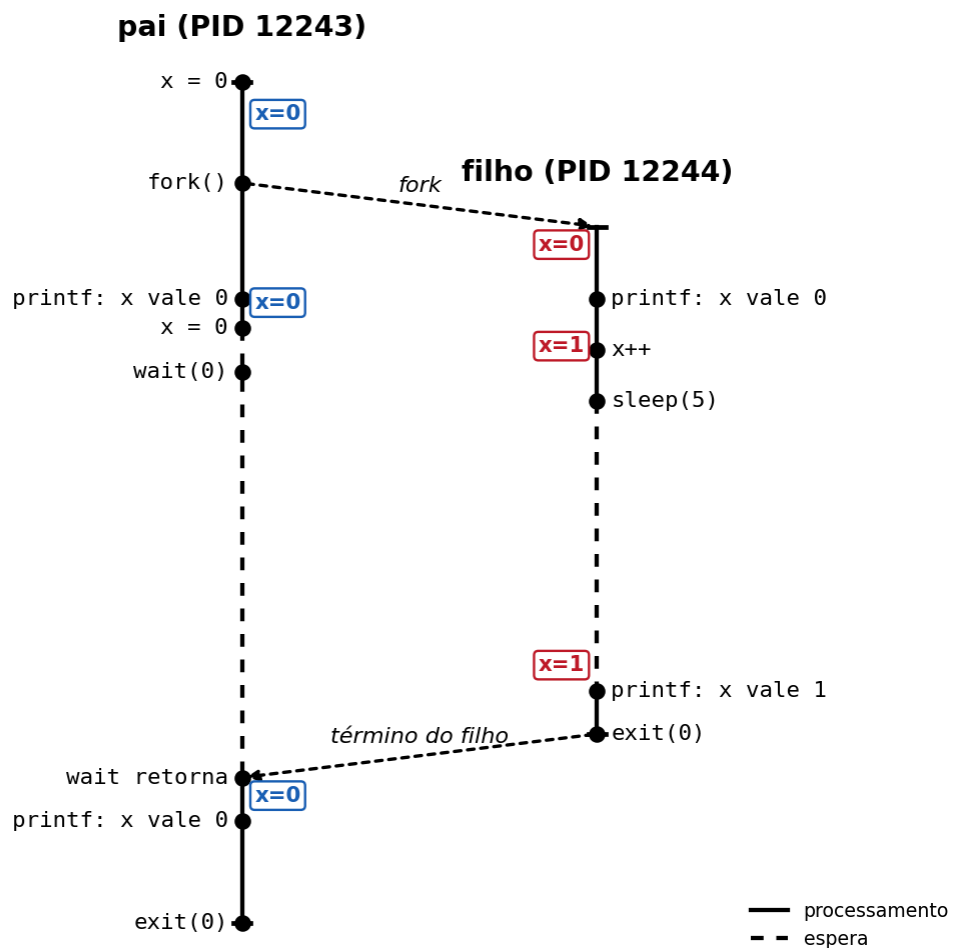
```

No processo 12243 x vale 0
No processo 12244 x vale 0
No processo 12244 x vale 1
No processo 12243 x vale 0

```

2. Diagrama de tempo com a evolução de x

Exercício 3 — fork-print.c (evolução de x; cada processo tem seu próprio x)



Resumo da evolução de x: no pai, $0 \rightarrow 0 \rightarrow 0$ (nunca muda); no filho, $0 \rightarrow 1$ (após `x++`). Os valores diferentes provam que não há memória compartilhada entre pai e filho após o `fork`.